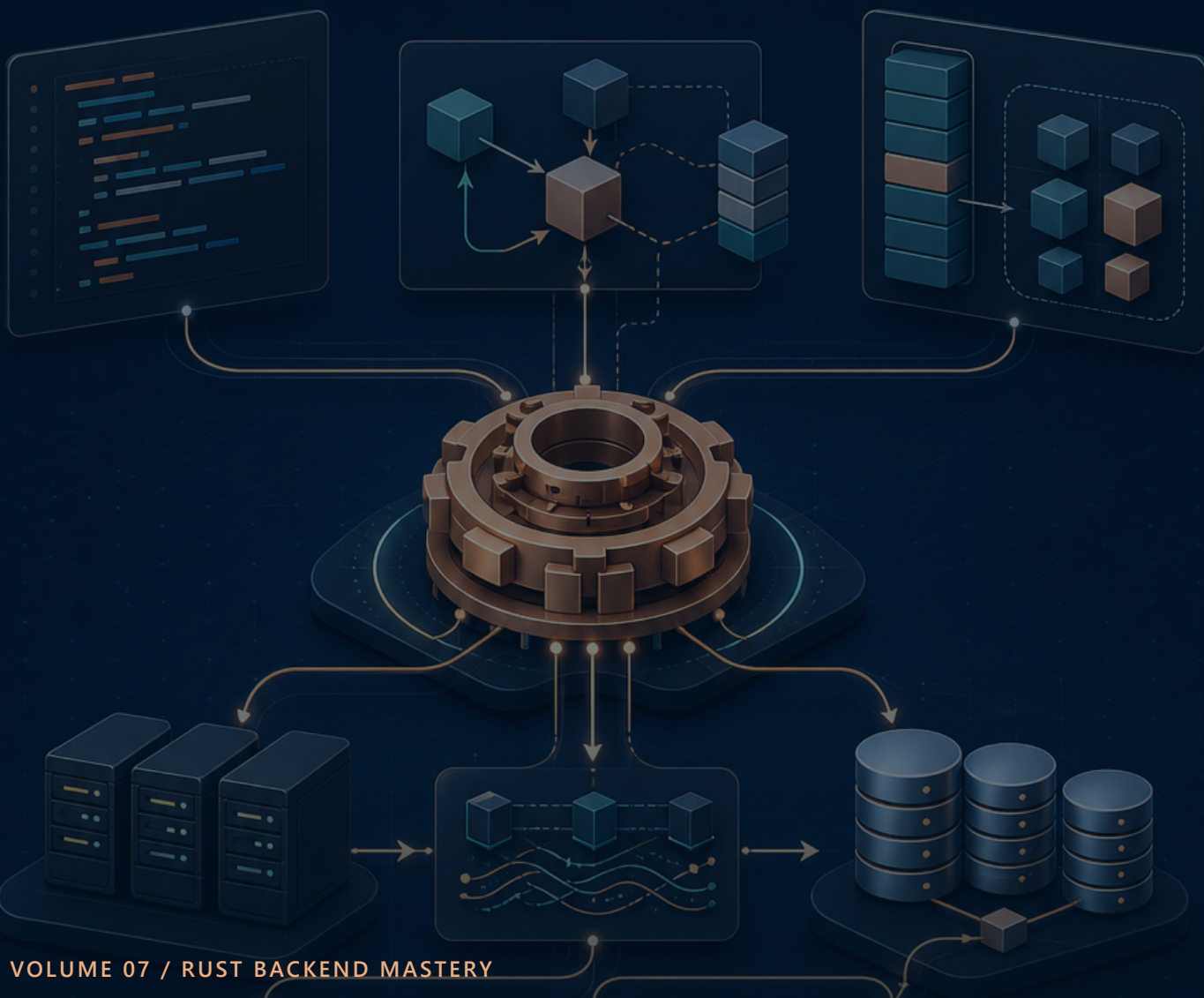




VOLUME 07 / RUST BACKEND MASTERY

Turso / libSQL — SQL設計・Rust実装・運用

SQLの意味、制約、index、transaction、migrationからlibsql接続と障害対応までを一続きで学ぶ

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

SDK呼び出しだけでなく、データモデルと一貫性を先に設計し、認可条件と運用手順をSQLへ落としします。

到達目標

- ✓ 制約を持つschemaを設計できる
- ✓ 安全なCRUDとindexを説明できる
- ✓ migrationとtransactionを運用できる
- ✓ libsql RepositoryとTurso障害対応を書ける

学習地図

01 関係モデルとCREATE TABLE

表は同じ形の行、列は型と意味、主キーは行の同一性、外部キーは表間の関係を表します。まず利用場面と不変条件を書いてからDDLへ落としします。

02 SELECT・WHERE・ORDER BY・LIMIT

SELECTは必要列だけを読み、WHEREで候補を絞り、ORDER BYで順序を決め、LIMITで返却量を制限します。順序なしの結果は不定であり、paginationには一意なtie-breakerが必要です。

03 INSERT・UPDATE・DELETEと所有者条件

更新系SQLは影響行数を確認します。UPDATE/DELETEのWHEREにidだけでなくuser_idを含めることで、他人のIDを指定しても変更できないDB境界の認可になります。

04 indexとEXPLAIN QUERY PLAN

indexは検索を速める別データ構造ですが、書込費用と容量を増やします。実際のWHERE・ORDER BYに合わせた複合indexを作り、EXPLAINで利用有無を確認します。

05 transaction・ACID・競合

transactionは複数SQLを全成功か全失敗にまとめます。残高移動、user削除と関連cleanup、job取得のように途中状態を見せてはいけない処理で使います。

06 migrationとschema version

migrationは適用済み順序を記録する前進可能なschema変更です。既存データがある前提で、nullable追加→backfill→制約強化のように段階化します。

07 libsql Rust SDK接続とparameter bind

libsql crateでBuilderからremote DBを作りconnectionを取得します。URLとtokenは環境変数から読み、SQL値はparams!でbindして文字列連結します。

08 local・remote・embedded replica

libSQLはローカルファイル、Turso remote、embedded replicaを選べます。replicaはlocal readの速度とoffline耐性を得ますが、sync間隔とread-your-writesを設計する必要があります。

09 Repository・row mapping・N+1

SQLをweb handlerへ散らさずRepositoryへ集め、rowからdomainへの変換失敗に列名とoperationの文脈を付けます。loop内queryによるN+1をjoinやINでまとめます。

10 backup・token・監視・障害対応

クラウドDBでもbackup、復元試験、token rotation、容量・latency・error監視が必要です。『自動backupがある』ではなくRPO/RTOと復元手順を確認します。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

tableとrow

STEP 2

primary key

STEP 3

foreign key

STEP 4

NOT NULLとCHECK

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

関係モデルとCREATE TABLE

UNIT 01・CONCEPT

関係モデルとCREATE TABLE

表は同じ形の行、列は型と意味、主キーは行の同一性、外部キーは表間の関係を表します。まず利用場面と不変条件を書いてからDDLへ落とします。

この単元の到達点

- tableとrow
- primary key
- foreign key
- NOT NULLとCHECK

なぜバックエンドで必要か

DB制約はアプリ検証の代わりではなく最後の防壁です。複数プロセスが同時更新しても一意性と参照整合性を守ります。



UNIT 01・MENTAL MODEL

関係モデルとCREATE TABLEを図でつかむ



表は同じ形の行、列は型と意味、主キーは行の同一性、外部キーは表間の関係を表します。まず利用場面と不変条件を書いてからDDLへ落とします。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	tableとrow	tableとrow — 入力と前提を確認し、境界を曖昧にしない。
2	primary key	primary key — 値がどの順で変換されるかを追跡する。
3	foreign key	foreign key — 失敗を再現し、型またはログから原因を特定する。
4	NOT NULLとCHECK	NOT NULLとCHECK — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

関係モデルとCREATE TABLEとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01 · MINIMUM CODE

まず動く最小コード

関係モデルとCREATE TABLEだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
CREATE TABLE users(id INTEGER PRIMARY KEY,email TEXT NOT NULL UNIQUE,created_at TEXT NOT NULL);
CREATE TABLE notes(id INTEGER PRIMARY KEY,user_id INTEGER NOT NULL REFERENCES users(id),title TEXT NOT NULL
CHECK(length(title) BETWEEN 1 AND 80),done INTEGER NOT NULL DEFAULT 0 CHECK(done IN(0,1)));
```

● ● ● 実行コマンド

```
turso db shell rustnote-dev
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
CREATE TABLE users(id INTEGER PRIMARY KEY,email TEXT NOT NULL UNIQUE,creat	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。
CREATE TABLE notes(id INTEGER PRIMARY KEY,user_id INTEGER NOT NULL REFEREN	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。
関係モデルとCREATE TABLE	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がtableとrowの条件を満たすか確認する。
- ✓ primary keyを実行し、途中の型と状態を追う。
- ✓ foreign keyで失敗を成功経路から分離する。
- ✓ NOT NULLとCHECKにより、出力と副作用を検証する。

型を声に出す 関係モデルとCREATE TABLEに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

schema created successfully

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
CREATE TABLE sessions(token_hash BLOB PRIMARY KEY,user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,expires_at TEXT NOT NULL,created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP);
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01・BACKEND APPLICATION

バックエンドのどこで使うか

DB制約はアプリ検証の代わりではなく最後の防壁です。複数プロセスが同時更新しても一意性と参照整合性を守ります。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	tableとrowに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	primary keyの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	foreign keyまたはNOT NULLとCHECKの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

関係モデルとCREATE TABLEを再現して切り分ける

関係モデルとCREATE TABLEの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: tableとrow
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: primary key
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: foreign key
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: NOT NULLとCHECK

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	tableとrow	tableとrow — 入力と前提を確認し、境界を曖昧にしない。
2	primary key	primary key — 値がどの順で変換されるかを追跡する。
3	foreign key	foreign key — 失敗を再現し、型またはログから原因を特定する。
4	NOT NULLとCHECK	NOT NULLとCHECK — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 要件を主キー・外部キー・NOT NULL・UNIQUE・CHECKへ変換します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 · PRACTICE

手を動かす演習

sessions表をtoken_hash主キー、user_id外部キー、expires_at必須で設計してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
CREATE TABLE sessions(token_hash BLOB PRIMARY KEY,user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,expires_at TEXT NOT NULL,created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP);
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 要件を主キー・外部キー・NOT NULL・UNIQUE・CHECKへ変換します。
- ✓ 関係モデルとCREATE TABLEとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

SELECT・WHERE・ORDER BY・LIMIT

UNIT 02・CONCEPT

SELECT・WHERE・ORDER BY・LIMIT

SELECTは必要列だけを読み、WHEREで候補を絞り、ORDER BYで順序を決め、LIMITで返却量を制限します。順序なしの結果は不定であり、paginationには一意なtie-breakerが必要です。

この単元の到達点

- 必要列だけSELECT
- WHERE条件
- 決定的ORDER BY
- LIMITとOFFSET

なぜバックエンドで必要か

必ずuser_idをWHEREへ含めて認可と検索を同時に行い、外部入力はplaceholder bindします。



UNIT 02・MENTAL MODEL

SELECT・WHERE・ORDER BY・LIMITを図でつかむ



SELECTは必要列だけを読み、WHEREで候補を絞り、ORDER BYで順序を決め、LIMITで返却量を制限します。順序なしの結果は不定であり、paginationには一意なtie-breakerが必要です。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	必要列だけSELECT	必要列だけSELECT — 入力と前提を確認し、境界を曖昧にしない。
2	WHERE条件	WHERE条件 — 値がどの順で変換されるかを追跡する。
3	決定的ORDER BY	決定的ORDER BY — 失敗を再現し、型またはログから原因を特定する。
4	LIMITとOFFSET	LIMITとOFFSET — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

SELECT・WHERE・ORDER BY・LIMITとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02・MINIMUM CODE

まず動く最小コード

SELECT・WHERE・ORDER BY・LIMITだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
SELECT id,title,done,created_at
FROM notes
WHERE user_id=?1 AND done=?2
ORDER BY created_at DESC,id DESC
LIMIT ?3 OFFSET ?4;
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
SELECT id,title,done,created_at	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。
FROM notes	この行が受け取る値、作る値、副作用の有無を確認します。
WHERE user_id=?1 AND done=?2	失敗時は早期returnし、呼び出し元へエラーを伝播します。
ORDER BY created_at DESC,id DESC	この行が受け取る値、作る値、副作用の有無を確認します。
LIMIT ?3 OFFSET ?4;	失敗時は早期returnし、呼び出し元へエラーを伝播します。

実行時に起きる順序

- ✓ 入力が必要列だけSELECTの条件を満たすか確認する。
- ✓ WHERE条件を実行し、途中の型と状態を追う。
- ✓ 決定的ORDER BYで失敗を成功経路から分離する。
- ✓ LIMITとOFFSETにより、出力と副作用を検証する。

型を声に出す SELECT・WHERE・ORDER BY・LIMITに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
id | title | done | created_at
7  | Rust  | 0    | 2026-08-11T...
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
SELECT id,title,done FROM notes WHERE user_id=?1 AND title LIKE '%||?2||%' ESCAPE '\\' ORDER BY id DESC
LIMIT MIN(?3,50);
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

必ずuser_idをWHEREへ含めて認可と検索を同時に行い、外部入力はplaceholder bindします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。

✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	必要列だけSELECTに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	WHERE条件の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	決定的ORDER BYまたはLIMITとOFFSETの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

SELECT・WHERE・ORDER BY・LIMITを再現して切り分ける

SELECT・WHERE・ORDER BY・LIMITの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 必要列だけSELECT
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: WHERE条件
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 決定的ORDER BY
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: LIMITとOFFSET

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	必要列だけSELECT	必要列だけSELECT — 入力と前提を確認し、境界を曖昧にしない。
2	WHERE条件	WHERE条件 — 値がどの順で変換されるかを追跡する。
3	決定的ORDER BY	決定的ORDER BY — 失敗を再現し、型またはログから原因を特定する。
4	LIMITとOFFSET	LIMITとOFFSET — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 SELECTは列・認可条件・順序・上限を明示し、値をbindします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

検索語をtitle LIKEで絞り、owner、順序、上限50を守るSQLを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
SELECT id,title,done FROM notes WHERE user_id=?1 AND title LIKE '%||?2||'%' ESCAPE '\ ' ORDER BY id DESC
LIMIT MIN(?3,50);
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ SELECTは列・認可条件・順序・上限を明示し、値をbindします。
- ✓ SELECT・WHERE・ORDER BY・LIMITとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

INSERT・UPDATE・DELETEと所有者条件

UNIT 03・CONCEPT

INSERT・UPDATE・DELETEと所有者条件

更新系SQLは影響行数を確認します。UPDATE/DELETEのWHEREにidだけでなくuser_idを含めることで、他人のIDを指定しても変更できないDB境界の認可になります。

この単元の到達点

- INSERT returning
- UPDATE条件
- DELETE条件
- rows affected

なぜバックエンドで必要か

rows_affected=0をNotFoundとして返し、存在確認を別SQLでして情報漏えいしないようにします。



UNIT 03・MENTAL MODEL

INSERT・UPDATE・DELETEと所有者条件を図でつかむ



更新系SQLは影響行数を確認します。UPDATE/DELETEのWHEREにidだけでなくuser_idを含めることで、他人のIDを指定しても変更できないDB境界の認可になります。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	INSERT returning	INSERT returning — 入力と前提を確認し、境界を曖昧にしない。
2	UPDATE条件	UPDATE条件 — 値がどの順で変換されるかを追跡する。
3	DELETE条件	DELETE条件 — 失敗を再現し、型またはログから原因を特定する。
4	rows affected	rows affected — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

INSERT・UPDATE・DELETEと所有者条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03 · MINIMUM CODE

まず動く最小コード

INSERT・UPDATE・DELETEと所有者条件だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
INSERT INTO notes(user_id,title) VALUES(?1,?2) RETURNING id,title,done;
UPDATE notes SET title=?1 WHERE id=?2 AND user_id=?3;
DELETE FROM notes WHERE id=?1 AND user_id=?2;
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
INSERT INTO notes(user_id,title) VALUES(?1,?2) RETURNING id,title,done;	失敗時は早期returnし、呼び出し元へエラーを伝播します。
UPDATE notes SET title=?1 WHERE id=?2 AND user_id=?3;	失敗時は早期returnし、呼び出し元へエラーを伝播します。
DELETE FROM notes WHERE id=?1 AND user_id=?2;	失敗時は早期returnし、呼び出し元へエラーを伝播します。

実行時に起きる順序

- ✓ 入力がINSERT returningの条件を満たすか確認する。
- ✓ UPDATE条件を実行し、途中の型と状態を追う。
- ✓ DELETE条件で失敗を成功経路から分離する。
- ✓ rows affectedにより、出力と副作用を検証する。

型を声に出す INSERT・UPDATE・DELETEと所有者条件に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
UPDATE rows_affected=1
DELETE rows_affected=0 → not found or not owned
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
UPDATE notes SET done=CASE done WHEN 0 THEN 1 ELSE 0 END,updated_at=CURRENT_TIMESTAMP WHERE id=?1 AND user_id=?2 RETURNING id,title,done;
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

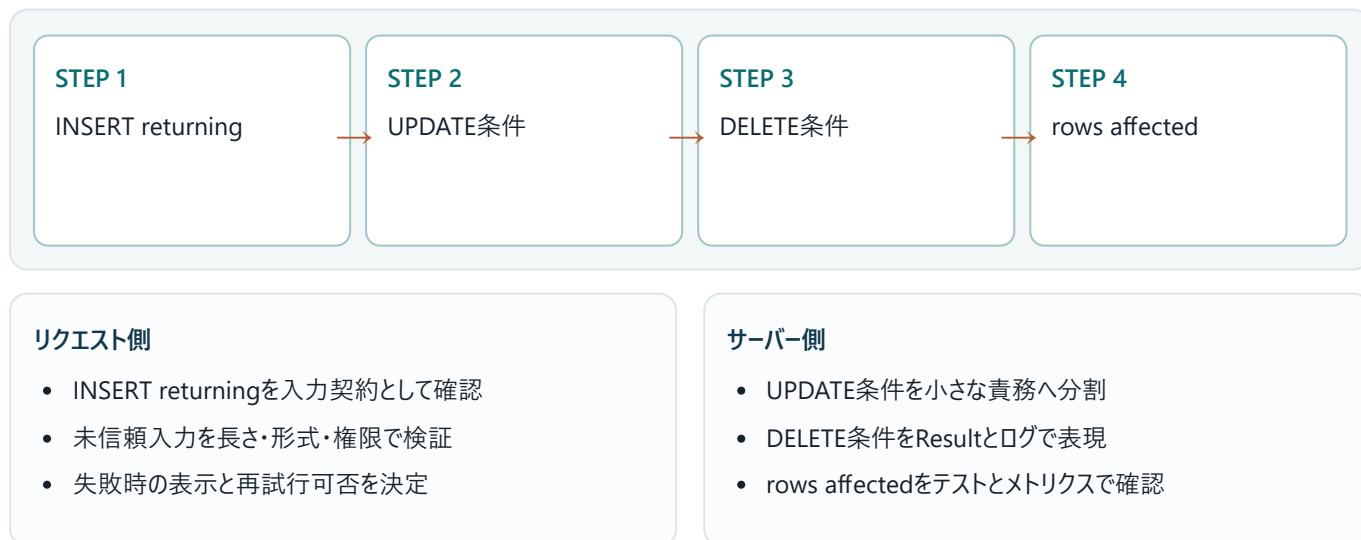
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 · BACKEND APPLICATION

バックエンドのどこで使うか

rows_affected=0をNotFoundとして返し、存在確認を別SQLでして情報漏えいしないようにします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。

✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	INSERT returningに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	UPDATE条件の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	DELETE条件またはrows affectedの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

INSERT・UPDATE・DELETEと所有者条件を再現して切り分ける

INSERT・UPDATE・DELETEと所有者条件の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: INSERT returning
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: UPDATE条件
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: DELETE条件
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: rows affected

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	INSERT returning	INSERT returning — 入力と前提を確認し、境界を曖昧にしない。
2	UPDATE条件	UPDATE条件 — 値がどの順で変換されるかを追跡する。
3	DELETE条件	DELETE条件 — 失敗を再現し、型またはログから原因を特定する。
4	rows affected	rows affected — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 更新SQL自体へowner条件を含め、影響行数で成功を判定します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

toggleをdone=CASE done WHEN 0 THEN 1 ELSE 0 ENDで一文UPDATEし、owner条件とRETURNINGを付けてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
UPDATE notes SET done=CASE done WHEN 0 THEN 1 ELSE 0 END,updated_at=CURRENT_TIMESTAMP WHERE id=?1 AND user_id=?2 RETURNING id,title,done;
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 更新SQL自体へowner条件を含め、影響行数で成功を判定します。
- ✓ INSERT・UPDATE・DELETEと所有者条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

indexとEXPLAIN QUERY PLAN

UNIT 04 · CONCEPT

indexとEXPLAIN QUERY PLAN

indexは検索を速める別データ構造ですが、書込費用と容量を増やします。実際のWHERE・ORDER BYに合わせた複合indexを作り、EXPLAINで利用有無を確認します。

この単元の到達点

- full scan
- B-tree index
- 複合列順
- EXPLAIN

なぜバックエンドで必要か

利用者ごとの新着一覧はuser_idを先頭にします。使われないindexを憶測で増やさず、query planと実測を残します。



UNIT 04 · MENTAL MODEL

indexとEXPLAIN QUERY PLANを図でつかむ



indexは検索を速める別データ構造ですが、書込費用と容量を増やします。実際のWHERE・ORDER BYに合わせた複合indexを作り、EXPLAINで利用有無を確認します。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果の種類と副作用を確認します。

順序	観察対象	初心者が確認すること
1	full scan	full scan — 入力と前提を確認し、境界を曖昧にしない。
2	B-tree index	B-tree index — 値がどの順で変換されるかを追跡する。
3	複合列順	複合列順 — 失敗を再現し、型またはログから原因を特定する。
4	EXPLAIN	EXPLAIN — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

indexとEXPLAIN QUERY PLANとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE まず動く最小コード

indexとEXPLAIN QUERY PLANだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
CREATE INDEX idx_notes_user_created ON notes(user_id,created_at DESC,id DESC);
EXPLAIN QUERY PLAN SELECT id,title FROM notes WHERE user_id=7 ORDER BY created_at DESC,id DESC LIMIT 20;
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
CREATE INDEX idx_notes_user_created ON notes(user_id,created_at DESC,id DE	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。

コード／場所	役割と理由
EXPLAIN QUERY PLAN SELECT id,title FROM notes WHERE user_id=7 ORDER BY cre	この行が受け取る値、作る値、副作用の有無を確認します。
indexとEXPLAIN QUERY PLAN	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がfull scanの条件を満たすか確認する。
- ✓ B-tree indexを実行し、途中の型と状態を追う。
- ✓ 複合列順で失敗を成功経路から分離する。
- ✓ EXPLAINにより、出力と副作用を検証する。

型を声に出す indexとEXPLAIN QUERY PLANに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

SEARCH notes USING INDEX idx_notes_user_created (user_id=?)

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
CREATE INDEX idx_notes_user_done_created ON notes(user_id,done,created_at DESC,id DESC);
EXPLAIN QUERY PLAN SELECT id,title FROM notes WHERE user_id=?1 AND done=?2 ORDER BY created_at DESC,id DESC
```

LIMIT 20;

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

利用者ごとの新着一覧はuser_idを先頭にします。使われないindexを憶測で増やさず、query planと実測を残します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	full scanに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	B-tree indexの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	複合列順またはEXPLAINの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

indexとEXPLAIN QUERY PLANを再現して切り分ける

indexとEXPLAIN QUERY PLANの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: full scan
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: B-tree index
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 複合列順
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: EXPLAIN

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	full scan	full scan — 入力と前提を確認し、境界を曖昧にしない。
2	B-tree index	B-tree index — 値がどの順で変換されるかを追跡する。
3	複合列順	複合列順 — 失敗を再現し、型またはログから原因を特定する。
4	EXPLAIN	EXPLAIN — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 indexはquery形に合わせ、EXPLAINと実測で効果を確認します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 ・ PRACTICE

手を動かす演習

doneで絞る一覧用に(user_id,done,created_at DESC,id DESC) indexを作りplanを比較してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
CREATE INDEX idx_notes_user_done_created ON notes(user_id,done,created_at DESC,id DESC);
EXPLAIN QUERY PLAN SELECT id,title FROM notes WHERE user_id=?1 AND done=?2 ORDER BY created_at DESC,id DESC
LIMIT 20;
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ indexはquery形に合わせ、EXPLAINと実測で効果を確認します。
- ✓ indexとEXPLAIN QUERY PLANとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

transaction・ACID・競合

UNIT 05・CONCEPT

transaction・ACID・競合

transactionは複数SQLを全成功か全失敗にまとめます。残高移動、user削除と関連cleanup、job取得のように途中状態を見せてはいけない処理で使います。

この単元の到達点

- BEGIN/COMMIT
- ROLLBACK
- 原子性
- 競合と短いtransaction

なぜバックエンドで必要か

transaction中に外部HTTPや長いCPU処理を行わず、DB操作だけを短くまとめてlock時間を減らします。



UNIT 05・MENTAL MODEL

transaction・ACID・競合を図でつかむ



transactionは複数SQLを全成功か全失敗にまとめます。残高移動、user削除と関連cleanup、job取得のように途中状態を見せてはいけない処理で使います。図では左から右へ処理を追います。各箱の入口では入力 of 型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	BEGIN/COMMIT	BEGIN/COMMIT — 入力と前提を確認し、境界を曖昧にしない。
2	ROLLBACK	ROLLBACK — 値がどの順で変換されるかを追跡する。
3	原子性	原子性 — 失敗を再現し、型またはログから原因を特定する。
4	競合と短いtransaction	競合と短いtransaction — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

transaction・ACID・競合とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05・MINIMUM CODE
まず動く最小コード

transaction・ACID・競合だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
BEGIN;  
UPDATE counters SET value=value+1 WHERE name='published';  
INSERT INTO audit_events(kind,note_id) VALUES('published',?1);  
COMMIT;
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05・LINE BY LINE
コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
BEGIN;	この行が受け取る値、作る値、副作用の有無を確認します。
UPDATE counters SET value=value+1 WHERE name='published';	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。
INSERT INTO audit_events(kind,note_id) VALUES('published',?1);	失敗時は早期returnし、呼び出し元へエラーを伝播します。
COMMIT;	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がBEGIN/COMMITの条件を満たすか確認する。
- ✓ ROLLBACKを実行し、途中の型と状態を追う。
- ✓ 原子性で失敗を成功経路から分離する。
- ✓ 競合と短いtransactionにより、出力と副作用を検証する。

型を声に出す transaction・ACID・競合に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

counter更新とaudit追加が同時に確定します。途中失敗なら両方戻ります。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
BEGIN;
INSERT INTO notes(user_id,title) VALUES(1,'transaction test');
INSERT INTO audit_events(kind,note_id) VALUES(NULL,last_insert_rowid()); -- NOT NULLで失敗
ROLLBACK;
SELECT * FROM notes WHERE title='transaction test';
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05・BACKEND APPLICATION

バックエンドのどこで使うか

transaction中に外部HTTPや長いCPU処理を行わず、DB操作だけを短くまとめてlock時間を減らします。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	BEGIN/COMMITに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	ROLLBACKの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	原子性または競合と短いtransactionの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

transaction・ACID・競合を再現して切り分ける

transaction・ACID・競合の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: BEGIN/COMMIT
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: ROLLBACK
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: 原子性
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 競合と短いtransaction

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	BEGIN/COMMIT	BEGIN/COMMIT — 入力と前提を確認し、境界を曖昧にしない。
2	ROLLBACK	ROLLBACK — 値がどの順で変換されるかを追跡する。
3	原子性	原子性 — 失敗を再現し、型またはログから原因を特定する。
4	競合と短いtransaction	競合と短いtransaction — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 整合性を守るSQLだけを短いtransactionへ入れ、外部I/Oを含めません。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

ノート作成と監査イベント追加をtransactionで実行し、二つ目を失敗させてrollbackを確認してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
BEGIN;  
INSERT INTO notes(user_id,title) VALUES(1,'transaction test');  
INSERT INTO audit_events(kind,note_id) VALUES(NULL,last_insert_rowid()); -- NOT NULLで失敗  
ROLLBACK;  
SELECT * FROM notes WHERE title='transaction test';
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 整合性を守るSQLだけを短いtransactionへ入れ、外部I/Oを含めません。
- ✓ transaction・ACID・競合とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

migrationとschema version

UNIT 06 · CONCEPT

migrationとschema version

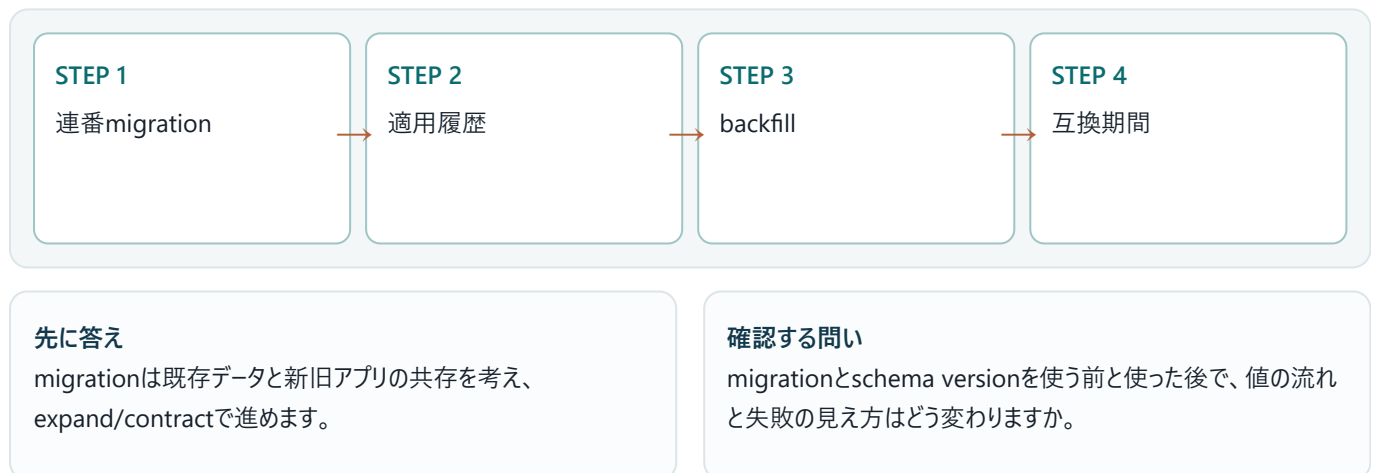
migrationは適用済み順序を記録する前進可能なschema変更です。既存データがある前提で、nullable追加→backfill→制約強化のように段階化します。

この単元の到達点

- 連番migration
- 適用履歴
- backfill
- 互換期間

なぜバックエンドで必要か

deploy前後に旧・新binaryが同時に動けるexpand/contractを使い、破壊的renameやdropを一度に行いません。



UNIT 06 · MENTAL MODEL

migrationとschema versionを図でつかむ



migrationは適用済み順序を記録する前進可能なschema変更です。既存データがある前提で、nullable追加→backfill→制約強化のように段階化します。図では左から右へ処理を追います。各箱の入口では入力のと信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	連番migration	連番migration — 入力と前提を確認し、境界を曖昧にしない。
2	適用履歴	適用履歴 — 値がどの順で変換されるかを追跡する。
3	backfill	backfill — 失敗を再現し、型またはログから原因を特定する。
4	互換期間	互換期間 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

migrationとschema versionとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06 · MINIMUM CODE まず動く最小コード

migrationとschema versionだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
-- 0002_add_updated_at.sql
ALTER TABLE notes ADD COLUMN updated_at TEXT;
UPDATE notes SET updated_at=created_at WHERE updated_at IS NULL;
-- 新旧アプリが共存する期間を経て制約を強化
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06 · LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
-- 0002_add_updated_at.sql	この行が受け取る値、作る値、副作用の有無を確認します。
ALTER TABLE notes ADD COLUMN updated_at TEXT;	この行が受け取る値、作る値、副作用の有無を確認します。
UPDATE notes SET updated_at=created_at WHERE updated_at IS NULL;	SQLの責務を一文で読み、プレースホルダと制約で入力を守ります。
-- 新旧アプリが共存する期間を経て制約を強化	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が連番migrationの条件を満たすか確認する。
- ✓ 適用履歴を実行し、途中の型と状態を追う。
- ✓ backfillで失敗を成功経路から分離する。
- ✓ 互換期間により、出力と副作用を検証する。

型を声に出す migrationとschema versionに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
migration 0002 applied
rows backfilled: 128
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 06 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
-- expand: ALTER TABLE notes ADD COLUMN archived_at TEXT;
-- deploy: new app writes/reads archived_at while old app ignores it
-- contract: after old app removed, add indexes or remove obsolete columns in later migration
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06 · BACKEND APPLICATION

バックエンドのどこで使うか

deploy前後に旧・新binaryが同時に動けるexpand/contractを使い、破壊的renameやdropを一度に行いません。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。

- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	連番migrationに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	適用履歴の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	backfillまたは互換期間の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

migrationとschema versionを再現して切り分ける

migrationとschema versionの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: 連番migration
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 適用履歴
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: backfill
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 互換期間

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	連番migration	連番migration — 入力と前提を確認し、境界を曖昧にしない。
2	適用履歴	適用履歴 — 値がどの順で変換されるかを追跡する。
3	backfill	backfill — 失敗を再現し、型またはログから原因を特定する。
4	互換期間	互換期間 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 migrationは既存データと新旧アプリの共存を考え、expand/contractで進めます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

notesへarchived_atを追加し、旧アプリと互換なmigration計画を三段階で書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
-- expand: ALTER TABLE notes ADD COLUMN archived_at TEXT;  
-- deploy: new app writes/reads archived_at while old app ignores it  
-- contract: after old app removed, add indexes or remove obsolete columns in later migration
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ migrationは既存データと新旧アプリの共存を考え、expand/contractで進めます。
- ✓ migrationとschema versionとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

libsql Rust SDK接続とparameter bind

UNIT 07 · CONCEPT

libsql Rust SDK接続とparameter bind

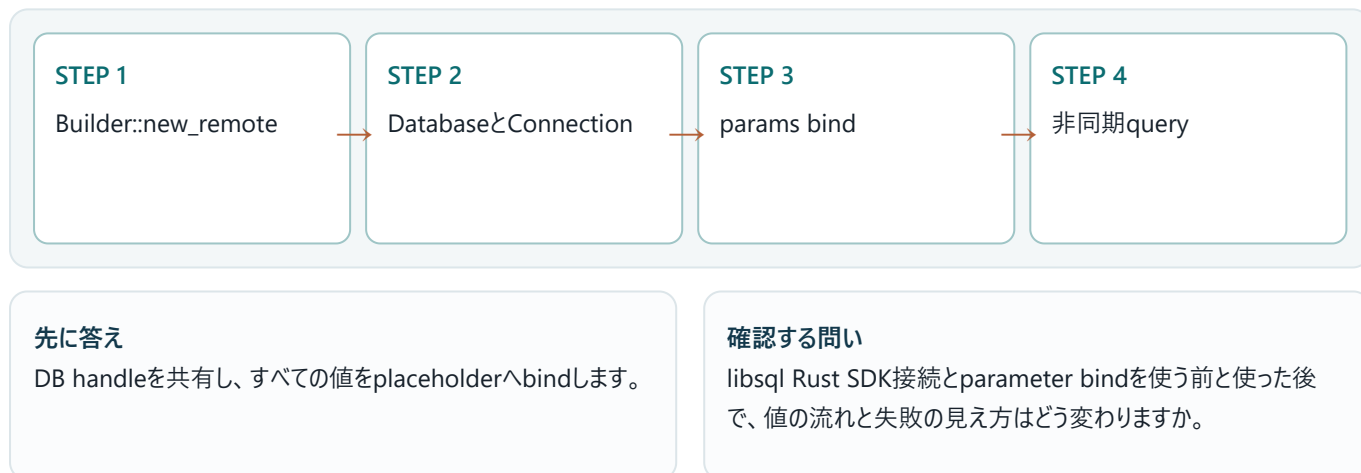
libsql crateでBuilderからremote DBを作りconnectionを取得します。URLとtokenは環境変数から読み、SQL値はparams!でbindして文字列連結しません。

この単元の到達点

- Builder::new_remote
- DatabaseとConnection
- params bind
- 非同期query

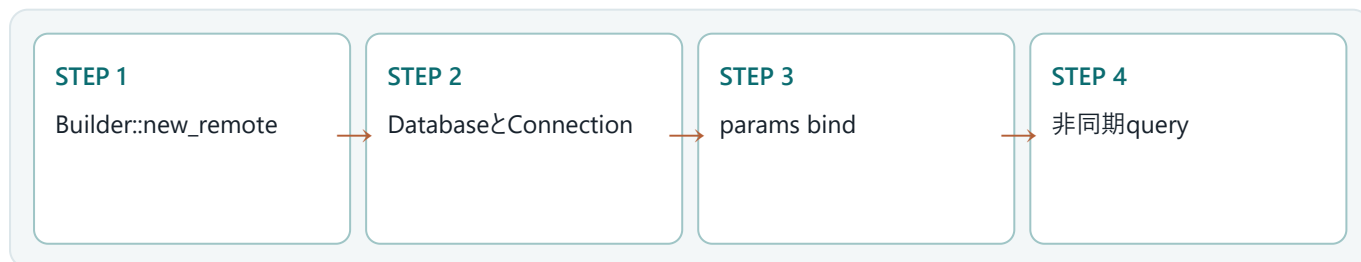
なぜバックエンドで必要か

connectionを毎requestで新規remote構築せず、Database handleをAppStateへ置きます。tokenをDebug出力しません。



UNIT 07 · MENTAL MODEL

libsql Rust SDK接続とparameter bindを図でつかむ



libsql crateでBuilderからremote DBを作りconnectionを取得します。URLとtokenは環境変数から読み、SQL値はparams!でbindして文字列連結しません。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Builder::new_remote	Builder::new_remote — 入力と前提を確認し、境界を曖昧にしない。
2	DatabaseとConnection	DatabaseとConnection — 値がどの順で変換されるかを追跡する。
3	params bind	params bind — 失敗を再現し、型またはログから原因を特定する。
4	非同期query	非同期query — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

libsql Rust SDK接続とparameter bindとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07・MINIMUM CODE まず動く最小コード

libsql Rust SDK接続とparameter bindだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
let db=libsql::Builder::new_remote(url,token).build().await?;
let conn=db.connect()?;
let mut rows=conn.query("SELECT id,title FROM notes WHERE user_id=?1 ORDER BY id DESC LIMIT ?
2",libsql::params![user_id,limit]).await?;
while let Some(row)=rows.next().await?{let id:i64=row.get(0)?;let title:String=row.get(1)?;}
```

●●● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>let db=libsql::Builder::new_remote(url,token).build().await?;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let conn=db.connect()?;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let mut rows=conn.query("SELECT id,title FROM notes WHERE user_id=?1 ORDER</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>while let Some(row)=rows.next().await?{let id:i64=row.get(0)?;let title:St</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。

実行時に起きる順序

- ✓ 入力がBuilder::new_remoteの条件を満たすか確認する。
- ✓ DatabaseとConnectionを実行し、途中の型と状態を追う。
- ✓ params bindで失敗を成功経路から分離する。
- ✓ 非同期queryにより、出力と副作用を検証する。

型を声に出す libsql Rust SDK接続とparameter bindに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

TursoへTLS接続し、bindされたuser_idの行だけを取得します。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
async fn insert(conn:&libsql::Connection,user_id:i64,title:&str)->libsql::Result<i64>{let mut
rows=conn.query("INSERT INTO notes(user_id,title) VALUES(?1,?2) RETURNING id",libsql::params!
[user_id,title]).await?;let row=rows.next().await?.ok_or(libsql::Error::QueryReturnedNoRows)?;row.get(0)}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

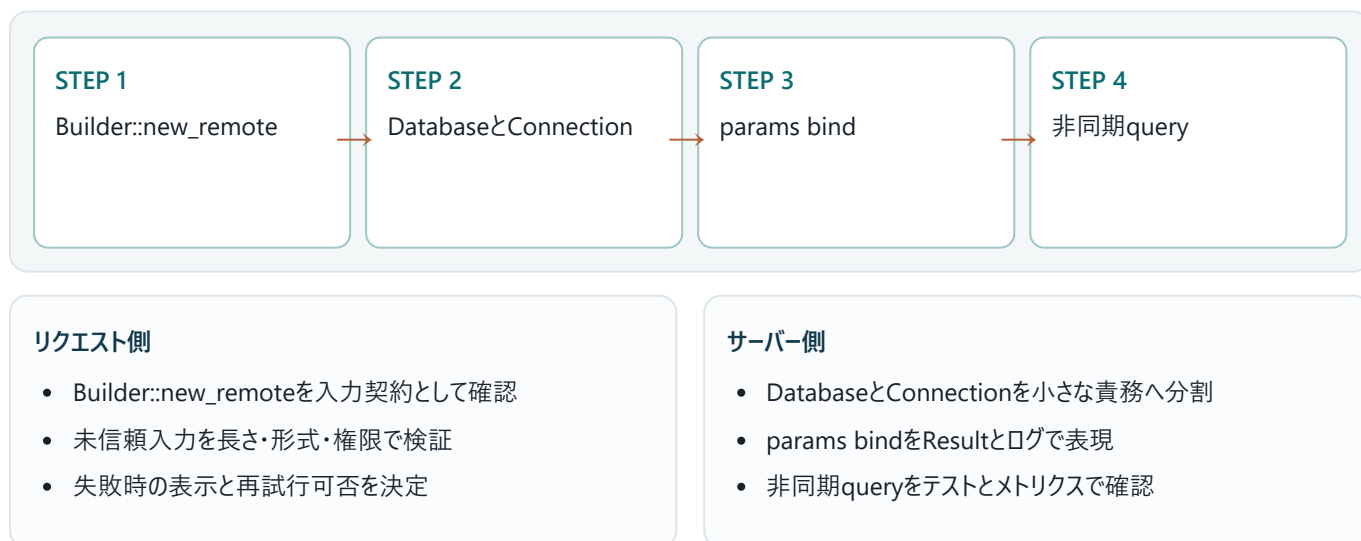
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07 · BACKEND APPLICATION

バックエンドのどこで使うか

connectionを毎requestで新規remote構築せず、Database handleをAppStateへ置きます。tokenをDebug出力しません。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Builder::new_remoteに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	DatabaseとConnectionの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	params bindまたは非同期queryの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

libsql Rust SDK接続とparameter bindを再現して切り分ける

libsql Rust SDK接続とparameter bindの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Builder::new_remote
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: DatabaseとConnection
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: params bind
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: 非同期query

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Builder::new_remote	Builder::new_remote — 入力と前提を確認し、境界を曖昧にしない。
2	DatabaseとConnection	DatabaseとConnection — 値がどの順で変換されるかを追跡する。
3	params bind	params bind — 失敗を再現し、型またはログから原因を特定する。
4	非同期query	非同期query — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 DB handleを共有し、すべての値をplaceholderへbindします。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

`insert_note`関数を作り、`title`をbindしRETURNING `id`を取得してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
async fn insert(conn:&libsql::Connection,user_id:i64,title:&str)->libsql::Result<i64>{let mut
rows=conn.query("INSERT INTO notes(user_id,title) VALUES(?1,?2) RETURNING id",libsql::params!
[user_id,title]).await?;let row=rows.next().await?.ok_or(libsql::Error::QueryReturnedNoRows)?;row.get(0)}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ DB handleを共有し、すべての値をplaceholderへbindします。
- ✓ libsql Rust SDK接続とparameter bindとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

local・remote・embedded replica

UNIT 08 ・ CONCEPT

local・remote・embedded replica

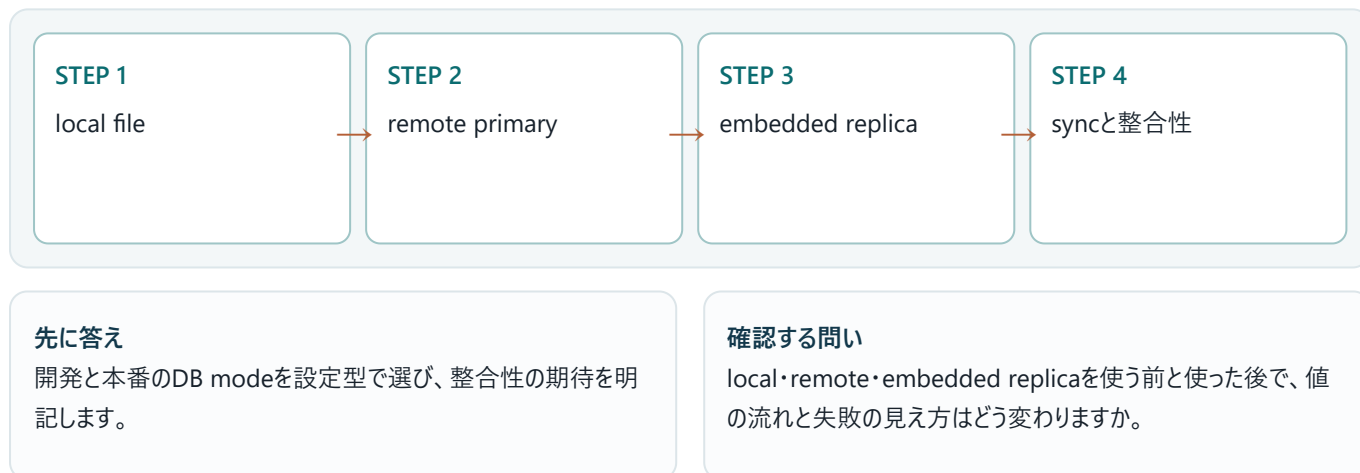
libSQLはローカルファイル、Turso remote、embedded replicaを選べます。replicaはlocal readの速度とoffline耐性を得ますが、sync間隔とread-your-writesを設計する必要があります。

この単元の到達点

- local file
- remote primary
- embedded replica
- syncと整合性

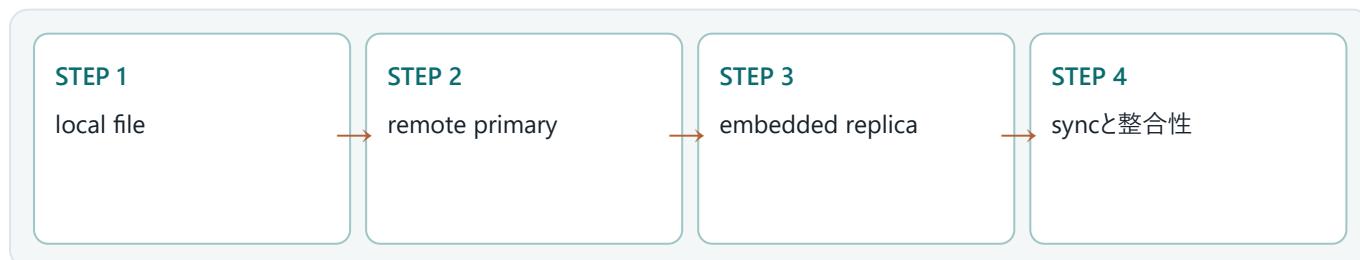
なぜバックエンドで必要か

Fly.ioのephemeral diskへ唯一のDBを置かずremote Tursoを主とします。replicaを使うならwrite後の同期と障害時の期待を明示します。



UNIT 08 ・ MENTAL MODEL

local・remote・embedded replicaを図でつかむ



libSQLはローカルファイル、Turso remote、embedded replicaを選べます。replicaはlocal readの速度とoffline耐性を得ますが、sync間隔とread-your-writesを設計する必要があります。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	local file	local file — 入力と前提を確認し、境界を曖昧にしない。
2	remote primary	remote primary — 値がどの順で変換されるかを追跡する。
3	embedded replica	embedded replica — 失敗を再現し、型またはログから原因を特定する。
4	syncと整合性	syncと整合性 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

local・remote・embedded replicaとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08・MINIMUM CODE まず動く最小コード

local・remote・embedded replicaだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
// local
let local=libsql::Builder::new_local("dev.db").build().await?;
// remote
let remote=libsql::Builder::new_remote(url,token).build().await?;
// replicaではsync_url/token/local pathを設定し、必要時db.sync().await?
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>let local=libsql::Builder::new_local("dev.db").build().await?;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>let remote=libsql::Builder::new_remote(url,token).build().await?;</code>	値へ名前を付けます。型推論に任せても、ここで何を所有するかを追います。
<code>local・remote・embedded replica</code>	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力がlocal fileの条件を満たすか確認する。
- ✓ remote primaryを実行し、途中の型と状態を追う。
- ✓ embedded replicaで失敗を成功経路から分離する。
- ✓ syncと整合性により、出力と副作用を検証する。

型を声に出す local・remote・embedded replicaに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

development: local dev.db
production: libsql://...turso.io

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
enum DbMode{Local{path:String},Remote{url:String,token:String}}
async fn open(m:DbMode)->libsql::Result<libsql::Database>{match
m{DbMode::Local{path}=>libsql::Builder::new_local(path).build().await,DbMode::Remote{url,token}=>libsql::Builder::new_remote(url,token).build().await}}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

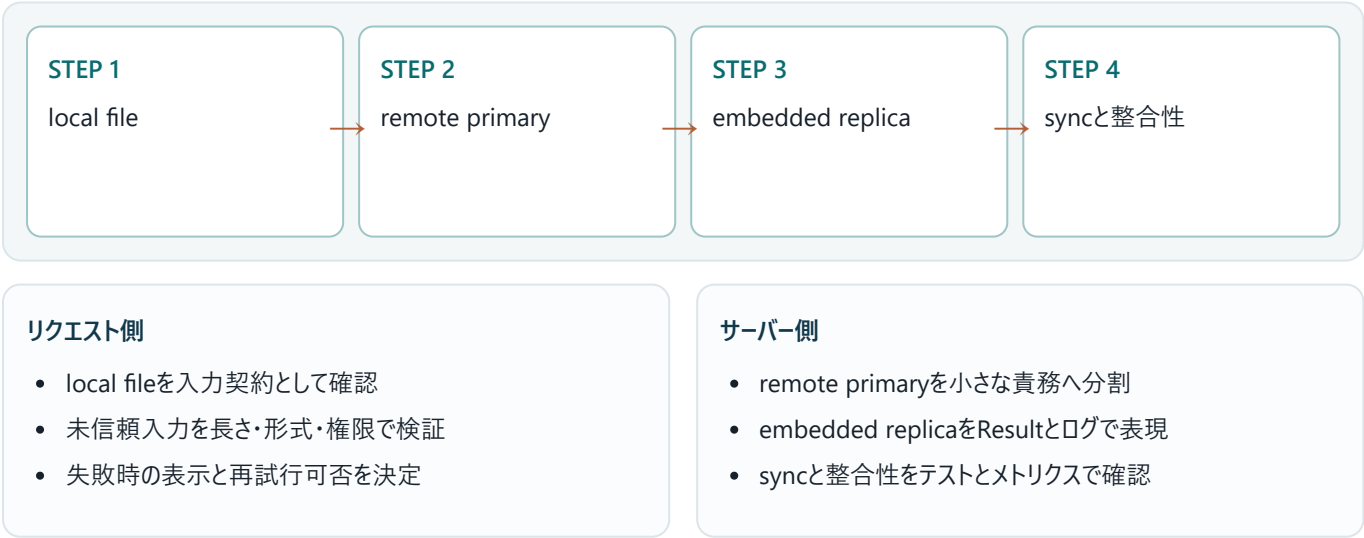
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08 · BACKEND APPLICATION

バックエンドのどこで使うか

Fly.ioのephemeral diskへ唯一のDBを置かずremote Tursoを主とします。replicaを使うならwrite後の同期と障害時の期待を明示します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	local fileに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	remote primaryの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	embedded replicaまたはsyncと整合性の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

local・remote・embedded replicaを再現して切り分ける

local・remote・embedded replicaの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: local file
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: remote primary
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: embedded replica
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: syncと整合性

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	local file	local file — 入力と前提を確認し、境界を曖昧にしない。
2	remote primary	remote primary — 値がどの順で変換されるかを追跡する。
3	embedded replica	embedded replica — 失敗を再現し、型またはログから原因を特定する。
4	syncと整合性	syncと整合性 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 開発と本番のDB modeを設定型で選び、整合性の期待を明記します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

APP_DB_MODE=local/remoteでBuilderを切り替えるConfigとfactoryの型設計を書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
enum DbMode{Local{path:String},Remote{url:String,token:String}}
async fn open(m:DbMode)->libsql::Result<libsql::Database>{match
m{DbMode::Local{path}=>libsql::Builder::new_local(path).build().await,DbMode::Remote{url,token}=>libsql::Builder::new_remote(url,token).build().await}}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 開発と本番のDB modeを設定型で選び、整合性の期待を明記します。
- ✓ local・remote・embedded replicaとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

Repository・row mapping・N+1

UNIT 09・CONCEPT

Repository・row mapping・N+1

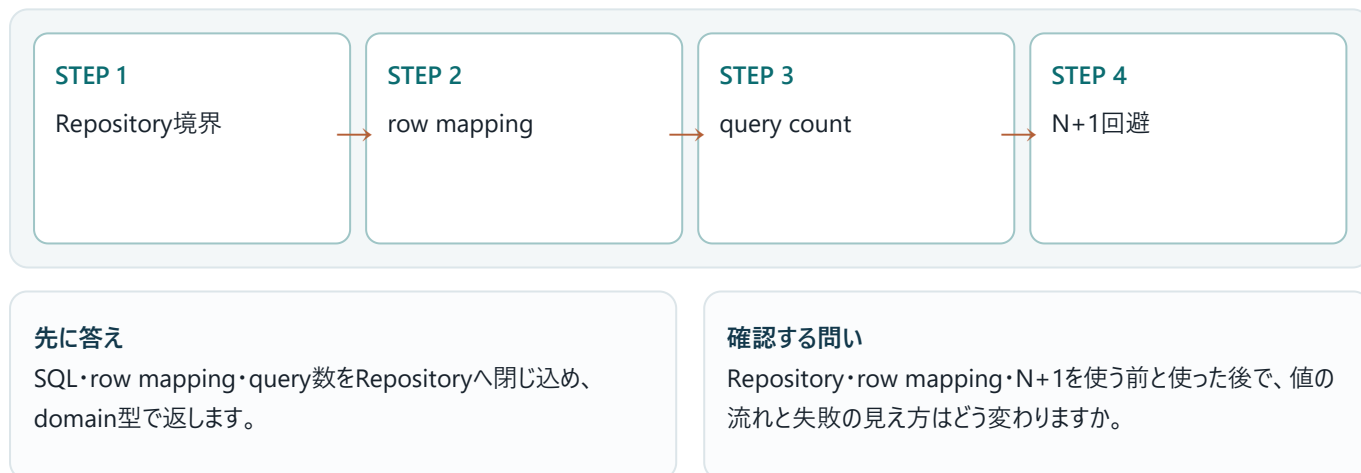
SQLをweb handlerへ散らさずRepositoryへ集め、rowからdomainへの変換失敗に列名とoperationの文脈を付けます。loop内queryによるN+1をjoinやINでまとめます。

この単元の到達点

- Repository境界
- row mapping
- query count
- N+1回避

なぜバックエンドで必要か

Repositoryの戻り値はlibsql Rowを外へ漏らさず、domain型または専用pageを返します。query回数をtrace fieldへ記録します。



UNIT 09・MENTAL MODEL

Repository・row mapping・N+1を図でつかむ



SQLをweb handlerへ散らさずRepositoryへ集め、rowからdomainへの変換失敗に列名とoperationの文脈を付けます。loop内queryによるN+1をjoinやINでまとめます。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	Repository境界	Repository境界 — 入力と前提を確認し、境界を曖昧にしない。
2	row mapping	row mapping — 値がどの順で変換されるかを追跡する。
3	query count	query count — 失敗を再現し、型またはログから原因を特定する。
4	N+1回避	N+1回避 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Repository・row mapping・N+1とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09・MINIMUM CODE
まず動く最小コード

Repository・row mapping・N+1だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
async fn list_notes(conn:&Connection,user_id:i64)->Result<Vec<Note>,RepoError>{let mut
rows=conn.query(SQL,params![user_id]).await?;let mut out=vec![];while let Some(row)=rows.next().await?
{out.push(Note{id:row.get(0)?,title:row.get(1)?,done:row.get::i64>(2)?!=0});}Ok(out)}
```

●●● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09・LINE BY LINE
コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
async fn list_notes(conn:&Connection,user_id:i64)->Result<Vec<Note>,RepoEr	関数は責務の境界です。引数が入力契約、戻り値が成功・失敗を含む出力契約になります。
Repository・row mapping・N+1	入力、処理、出力を分離して読みます。
Repository・row mapping・N+1	入力、処理、出力を分離して読みます。

実行時に起きる順序

- ✓ 入力Repository境界の条件を満たすか確認する。
- ✓ row mappingを実行し、途中の型と状態を追う。
- ✓ query countで失敗を成功経路から分離する。
- ✓ N+1回避により、出力と副作用を検証する。

型を声に出す Repository・row mapping・N+1に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

一回のqueryで所有者のNote一覧をVec<Note>へ変換します。

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
SELECT u.id,u.email,COUNT(n.id) AS note_count FROM users u LEFT JOIN notes n ON n.user_id=u.id WHERE u.id=?1
GROUP BY u.id,u.email;
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

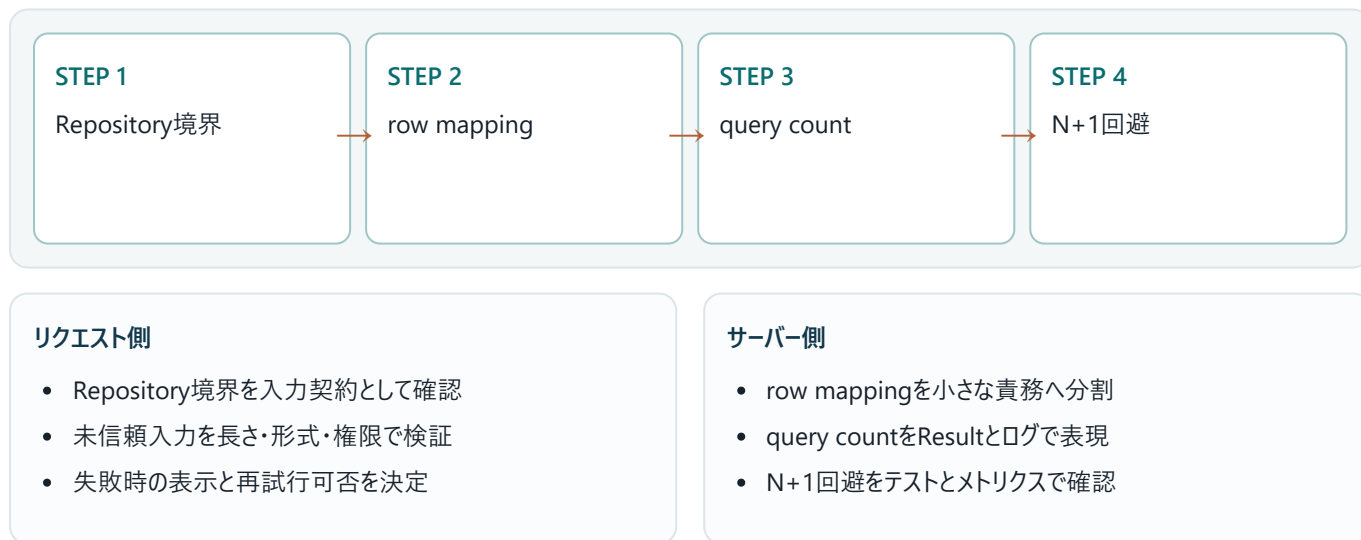
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09 ・ BACKEND APPLICATION

バックエンドのどこで使うか

Repositoryの戻り値はlibsql Rowを外へ漏らさず、domain型または専用pageを返します。query回数をtrace fieldへ記録します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	Repository境界に関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	row mappingの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	query countまたはN+1回避の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

Repository・row mapping・N+1を再現して切り分ける

Repository・row mapping・N+1の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: Repository境界
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: row mapping
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: query count
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: N+1回避

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	Repository境界	Repository境界 — 入力と前提を確認し、境界を曖昧にしない。
2	row mapping	row mapping — 値がどの順で変換されるかを追跡する。
3	query count	query count — 失敗を再現し、型またはログから原因を特定する。
4	N+1回避	N+1回避 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 SQL・row mapping・query数をRepositoryへ閉じ込め、domain型で返します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

userとnote件数を一回のLEFT JOIN/GROUP BYで取得するSQLを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
SELECT u.id,u.email,COUNT(n.id) AS note_count FROM users u LEFT JOIN notes n ON n.user_id=u.id WHERE u.id=?1  
GROUP BY u.id,u.email;
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ SQL・row mapping・query数をRepositoryへ閉じ込め、domain型で返します。
- ✓ Repository・row mapping・N+1とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

backup・token・監視・障害対応

UNIT 10・CONCEPT

backup・token・監視・障害対応

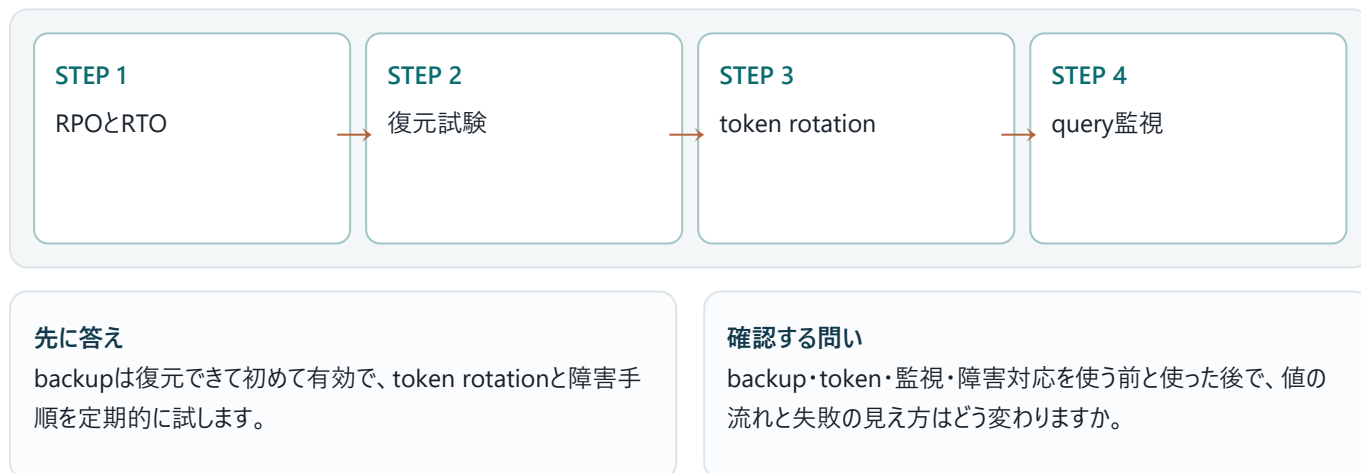
クラウドDBでもbackup、復元試験、token rotation、容量・latency・error監視が必要です。『自動backupがある』ではなくRPO/RTOと復元手順を確認します。

この単元の到達点

- RPOとRTO
- 復元試験
- token rotation
- query監視

なぜバックエンドで必要か

DB障害時にHTTP 500を乱発せず、503分類、timeout、circuit的抑制、read-only fallbackの可否を決めます。



UNIT 10・MENTAL MODEL

backup・token・監視・障害対応を図でつかむ



クラウドDBでもbackup、復元試験、token rotation、容量・latency・error監視が必要です。『自動backupがある』ではなくRPO/RTOと復元手順を確認します。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	RPOとRTO	RPOとRTO — 入力と前提を確認し、境界を曖昧にしない。
2	復元試験	復元試験 — 値がどの順で変換されるかを追跡する。
3	token rotation	token rotation — 失敗を再現し、型またはログから原因を特定する。
4	query監視	query監視 — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

backup・token・監視・障害対応とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

backup・token・監視・障害対応だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
# 秘密は表示せず更新
fly secrets set TURSO_DATABASE_URL=libsql://...
fly secrets set TURSO_AUTH_TOKEN=<new-token>
# 新tokenで疎通後、旧tokenをrevoke
```

● ● ● 実行コマンド

```
fly logs
fly status
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
# 秘密は表示せず更新	この行が受け取る値、作る値、副作用の有無を確認します。
fly secrets set TURSO_DATABASE_URL=libsql://...	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly secrets set TURSO_AUTH_TOKEN=<new-token>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
# 新tokenで疎通後、旧tokenをrevoke	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がRPOとRTOの条件を満たすか確認する。
- ✓ 復元試験を実行し、途中の型と状態を追う。
- ✓ token rotationで失敗を成功経路から分離する。
- ✓ query監視により、出力と副作用を検証する。

型を声に出す backup・token・監視・障害対応に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
new secret deployed
health check: database ok
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

1. db_latencyとerror分類を確認
 2. /healthのDB依存と利用者影響を確認
 3. timeout短縮・負荷制限・必要なら機能停止
 4. read/write疎通と未処理jobを確認
 5. query plan・容量・token・再発防止テストを更新

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10・BACKEND APPLICATION

バックエンドのどこで使うか

DB障害時にHTTP 500を乱発せず、503分類、timeout、circuit的抑制、read-only fallbackの可否を決めます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	RPOとRTOに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	復元試験の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	token rotationまたはquery監視の環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

backup・token・監視・障害対応を再現して切り分ける

backup・token・監視・障害対応の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: RPOとRTO
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 復元試験
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: token rotation
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: query監視

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	RPOとRTO	RPOとRTO — 入力と前提を確認し、境界を曖昧にしない。
2	復元試験	復元試験 — 値がどの順で変換されるかを追跡する。
3	token rotation	token rotation — 失敗を再現し、型またはログから原因を特定する。
4	query監視	query監視 — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 backupは復元できて初めて有効で、token rotationと障害手順を定期的に試します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

DB timeout時のrunbookを検知、影響確認、緩和、復旧確認、事後対応の五段階で書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

1. db_latencyとerror分類を確認
2. /healthのDB依存と利用者影響を確認
3. timeout短縮・負荷制限・必要なら機能停止
4. read/write疎通と未処理jobを確認
5. query plan・容量・token・再発防止テストを更新

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ backupは復元できて初めて有効で、token rotationと障害手順を定期的に試します。
- ✓ backup・token・監視・障害対応とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

Turso上へ認可条件付きNoteRepositoryを構築する

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	関係モデルとCREATE TABLE	要件を主キー・外部キー・NOT NULL・UNIQUE・CHECKへ変換します。
02	SELECT・WHERE・ORDER BY・LIMIT	SELECTは列・認可条件・順序・上限を明示し、値をbindします。
03	INSERT・UPDATE・DELETEと所有者条件	更新SQL自体へowner条件を含め、影響行数で成功を判定します。
04	indexとEXPLAIN QUERY PLAN	indexはquery形に合わせ、EXPLAINと実測で効果を確認します。
05	transaction・ACID・競合	整合性を守るSQLだけを短いtransactionへ入れ、外部I/Oを含めません。
06	migrationとschema version	migrationは既存データと新旧アプリの共存を考え、expand/contractで進めます。
07	libsql Rust SDK接続とparameter bind	DB handleを共有し、すべての値をplaceholderへbindします。
08	local・remote・embedded replica	開発と本番のDB modeを設定型で選び、整合性の期待を明記します。
09	Repository・row mapping・N+1	SQL・row mapping・query数をRepositoryへ閉じ込め、domain型で返します。
10	backup・token・監視・障害対応	backupは復元できて初めて有効で、token rotationと障害手順を定期的に試します。

完了条件

- ✓ 全SQLがbind parameterを使う
- ✓ owner条件と制約がDBにもある
- ✓ migrationが新旧互換である
- ✓ backup・token rotation・timeout手順がある

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. 関係モデルとCREATE TABLEを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. SELECT・WHERE・ORDER BY・LIMITを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. INSERT・UPDATE・DELETEと所有者条件を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. indexとEXPLAIN QUERY PLANを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. transaction・ACID・競合を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. migrationとschema versionを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. libsql Rust SDK接続とparameter bindを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. local・remote・embedded replicaを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. Repository・row mapping・N+1を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. backup・token・監視・障害対応を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. 関係モデルとCREATE TABLE

関係モデルとCREATE TABLEとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 要件を主キー・外部キー・NOT NULL・UNIQUE・CHECKへ変換します。

02. SELECT・WHERE・ORDER BY・LIMIT

SELECT・WHERE・ORDER BY・LIMITとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: SELECTは列・認可条件・順序・上限を明示し、値をbindします。

03. INSERT・UPDATE・DELETEと所有者条件

INSERT・UPDATE・DELETEと所有者条件とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 更新SQL自体へowner条件を含め、影響行数で成功を判定します。

04. indexとEXPLAIN QUERY PLAN

indexとEXPLAIN QUERY PLANとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: indexはquery形に合わせ、EXPLAINと実測で効果を確認します。

05. transaction・ACID・競合

transaction・ACID・競合とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 整合性を守るSQLだけを短いtransactionへ入れ、外部I/Oを含めません。

06. migrationとschema version

migrationとschema versionとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: migrationは既存データと新旧アプリの共存を考え、expand/contractで進めます。

07. libsql Rust SDK接続とparameter bind

libsql Rust SDK接続とparameter bindとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: DB handleを共有し、すべての値をplaceholderへbindします。

08. local・remote・embedded replica

local・remote・embedded replicaとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 開発と本番のDB modeを設定型で選び、整合性の期待を明記します。

09. Repository・row mapping・N+1

Repository・row mapping・N+1とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: SQL・row mapping・query数をRepositoryへ閉じ込め、domain型で返します。

10. backup・token・監視・障害対応

backup・token・監視・障害対応とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: backupは復元できて初めて有効で、token rotationと障害手順を定期的に試します。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
関係モデルとCREATE TABLE	要件を主キー・外部キー・NOT NULL・UNIQUE・CHECKへ変換します。	DB制約はアプリ検証の代わりではなく最後の防壁です。複数プロセスが同時更新しても一意性と参照整合性を守ります。
SELECT・WHERE・ORDER BY・LIMIT	SELECTは列・認可条件・順序・上限を明示し、値をbindします。	必ずuser_idをWHEREへ含めて認可と検索を同時に行い、外部入力はplaceholder bindします。
INSERT・UPDATE・DELETEと所有者条件	更新SQL自体へowner条件を含め、影響行数で成功を判定します。	rows_affected=0をNotFoundとして返し、存在確認を別SQLでして情報漏えいしないようにします。
indexとEXPLAIN QUERY PLAN	indexはquery形に合わせ、EXPLAINと実測で効果を確認します。	利用者ごとの新着一覧はuser_idを先頭にします。使われないindexを臆測で増やさず、query planと実測を残します。
transaction・ACID・競合	整合性を守るSQLだけを短いtransactionへ入れ、外部I/Oを含めません。	transaction中に外部HTTPや長いCPU処理を行わず、DB操作だけを短くまとめてlock時間を減らします。
migrationとschema version	migrationは既存データと新旧アプリの共存を考え、expand/contractで進めます。	deploy前後に旧・新binaryが同時に動けるexpand/contractを使い、破壊的renameやdropを一度に行いません。
libsql Rust SDK接続とparameter bind	DB handleを共有し、すべての値をplaceholderへbindします。	connectionを毎requestで新規remote構築せず、Database handleをAppStateへ置きます。tokenをDebug出力しません。
local・remote・embedded replica	開発と本番のDB modeを設定型で選び、整合性の期待を明記します。	Fly.ioのephemeral diskへ唯一のDBを置かずremote Tursoを主とします。replicaを使うならwrite後の同期と障害時の期待を明示します。
Repository・row mapping・N+1	SQL・row mapping・query数をRepositoryへ閉じ込め、domain型で返します。	Repositoryの戻り値はlibsql Rowを外へ漏らさず、domain型または専用pageを返します。query回数をtrace fieldへ記録します。
backup・token・監視・障害対応	backupは復元できて初めて有効で、token rotationと障害手順を定期的に試します。	DB障害時にHTTP 500を乱発せず、503分類、timeout、 circuit 的抑制、read-only fallbackの可否を決めます。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。